Amazon RDS for PostgreSQL DBA Daily Activities and Real-Time Scenarios

Day-to-day routines, periodic tasks, incident playbooks with diagnose-and-fix steps, change procedures, and quick answers

Version 1.0 | 29 October 2026

About this guide

This guide describes what a PostgreSQL DBA actually does on Amazon RDS, in the order the work arrives: a **daily routine**, **weekly/monthly/quarterly** upkeep, **real-time incidents** (what you see, what to run, how to read it, what to fix), **routine change tasks**, and **quick answers** to common questions.

Every item uses the same layout so it can be followed under pressure: purpose or symptom, the exact commands (placeholders in angle brackets), how to interpret the result, the action to take, and how to prevent a repeat. Items tagged **[HIGH IMPACT]** can cause an outage or data loss and deserve extra care.

Notes: SQL runs in `psql` as a member of `rds_superuser`; AWS commands use AWS CLI v2. Thresholds are suggested starting points, so tune them to your workload. Some statistic column names differ by version (for example `total_exec_time` in `pg_stat_statements` is PostgreSQL 13+). Test anything unfamiliar on a non-production clone first.

Who does what on RDS

AWS manages	You manage (the DBA's job)
Hardware, host OS and patching of the underlying fleet; automated backups and point-in-time recovery mechanics; Multi-AZ failover mechanics; engine binaries (when you schedule upgrades)	Schema, queries and indexes; vacuum and bloat health; parameter tuning; users, grants and security groups; capacity and cost; monitoring and alerting; restore and failover testing; how applications connect and retry
Not available to you: OS/SSH access, a real superuser (you have `rds_superuser`), editing pg_hba.conf, unsupported extensions, direct file-system access	Use instead: CloudWatch, Enhanced Monitoring, Performance Insights, RDS events, log export, parameter groups, `aws_s3` for file import/export

Metrics to watch and suggested alert thresholds

Metric	What it tells you	Warning	Critical
CPUUtilization	Compute saturation	> 80% for 15 min	> 90% for 10 min
DatabaseConnections	Closeness to `max_connections`	> 80% of limit	> 90% of limit
FreeableMemory / SwapUsage	Memory pressure, risk of OOM restart	< 20% free, swap rising	< 10% free, swap growing
FreeStorageSpace	Disk headroom	< 20% free	< 10% free
Read/WriteLatency	Storage responsiveness	> 10 ms sustained	> 25 ms sustained
DiskQueueDepth	I/O queueing	Rising with latency	Sustained high
ReplicaLag	Replica freshness	> 30 s	> 300 s
MaximumUsedTransactionIDs	Wraparound risk	> 500 million	> 1 billion
OldestReplicationSlotLag, TransactionLogsDiskUsage	Slot retaining WAL	> 10% of storage	> 25% of storage
BurstBalance (gp2), CPUCreditBalance (T classes)	Burst credits left	< 50%	< 20%

A day in the life of an RDS PostgreSQL DBA

When	Activity	Items
Start of day (30 min)	Morning health check: events, metrics, backups, sessions, top SQL, vacuum, logs, replication, storage, alarms	D-01 to D-11
During the day	Respond to alarms and tickets; incident handling; review and tune slow queries; grant access; support releases and schema changes	RT-01 to RT-18, W-04, C-01
Before and after deployments	Review DDL for locking risk; watch CPU, locks and top SQL afterwards; keep a rollback path	RT-03, RT-04
Evening / off-peak	Heavy maintenance: index builds, vacuum freeze, pg_repack, batch tuning, planned changes in the maintenance window	W-03, C-03, C-05
Weekly	Capacity, indexes, bloat, slow queries, security review, parameter drift	W-01 to W-06
Monthly	Restore test, patch planning, failover drill, certificates, cost review	M-01 to M-05
Quarterly	Version lifecycle, DR test, access recertification	Q-01

Contents at a glance

Part 1: Daily routine

ID	Item	ID	Item
D-01	Instance status and last 24 h of events	D-07	Autovacuum and dead tuples
D-02	Key CloudWatch metrics (last 24 h)	D-08	Error log review
D-03	Backups are running and restorable	D-09	Replication: read replicas and slots
D-04	Connections and session states	D-10	Storage and growth
D-05	Long-running queries, transactions and blocked sessions	D-11	Alarms, pending maintenance, open tickets
D-06	Top SQL by total time		

Part 2: Weekly, monthly, quarterly

ID	Item	ID	Item
W-01	WEEKLY: Capacity and growth review	M-01	MONTHLY: Restore test (point-in-time recovery to a scratch instance)
W-02	WEEKLY: Index review (unused, duplicate, missing)	M-02	MONTHLY: Minor version and maintenance planning
W-03	WEEKLY: Bloat check and autovacuum tuning	M-03	MONTHLY: Failover drill (Multi-AZ)
W-04	WEEKLY: Slow-query tuning cycle	M-04	MONTHLY: CA certificate and TLS expiry
W-05	WEEKLY: Access and security review	M-05	MONTHLY: Cost and right-sizing
W-06	WEEKLY: Parameter and configuration drift	Q-01	QUARTERLY: Governance, DR and lifecycle

Part 3: Real-time scenarios

ID	Item	ID	Item
RT-01	CPU at 90 to 100%	RT-10	Unexpected failover or reboot
RT-02	Too many connections	RT-11	Accidental DELETE, UPDATE or DROP: recover the data
RT-03	One query suddenly became slow (plan regression)	RT-12	Deadlocks
RT-04	Lock pile-up: sessions waiting on a blocker	RT-13	Memory pressure: low FreeableMemory, swap, temp-file spills
RT-05	Idle-in-transaction and very old transactions	RT-14	Inactive replication slot is retaining WAL
RT-06	Free storage dropping fast or storage full	RT-15	High I/O latency or DiskQueueDepth
RT-07	Read replica lag keeps growing	RT-16	Cannot connect to the database
RT-08	Autovacuum cannot keep up: dead tuples and bloat	RT-17	Parameter change did not take effect
RT-09	Transaction ID wraparound warning	RT-18	'permission denied' for a new or existing user

Part 4: Change tasks

ID	Item	ID	Item
C-01	Create application roles (read-only and read-write)	C-05	Minor version upgrade
C-02	Change a parameter safely	C-06	Major version upgrade (Blue/Green recommended)
C-03	Scale instance class, storage or IOPS	C-07	Logical backup and data export
C-04	Create and use a read replica	C-08	Rotate credentials

Part 5: Quick answers (23 questions) | Part 6: Command cheat sheet

PART 1 | Daily routine (morning health check)

Work through these in order each morning, about 30 minutes. They are read-only, so they are safe to run on production. Record anything unusual in your handover notes, and follow the scenario reference when a check is not healthy.

D-01 Instance status and last 24 h of events

Purpose

The first check of the day. RDS records failovers, reboots, storage changes, backups and maintenance as events, so one command tells you what happened overnight.

Run

```
aws rds describe-db-instances --db-instance-identifier <id> \
  --query "DBInstances[0].[DBInstanceStatus,MultiAZ,DBInstanceClass,
           EngineVersion,PendingModifiedValues]"

aws rds describe-events --source-type db-instance --source-identifier <id> \
  --duration 1440 --query "Events[].[Date,Message]" --output table
```

Healthy looks like

Status is `available`, `PendingModifiedValues` is empty, and events show only routine items (backup started/finished).

If not

Status such as `storage-full`, `incompatible-parameters` or `failed`, or an unexpected failover/reboot event: go to RT-10 or RT-06 and raise an incident.

D-02 Key CloudWatch metrics (last 24 h)

Purpose

Trends matter more than single values. Look at the same set of metrics every day so that 'unusual' becomes obvious. Thresholds are in the table on the previous pages.

Run

```
aws cloudwatch get-metric-statistics --namespace AWS/RDS --metric-name CPUUtilization \
  --dimensions Name=DBInstanceIdentifier,Value=<id> \
  --start-time $(date -u -d '-24 hours' +%FT%TZ) --end-time $(date -u +%FT%TZ) \
  --period 3600 --statistics Average Maximum --output table

# Review the same way (or on one dashboard): DatabaseConnections, FreeableMemory,
# SwapUsage, FreeStorageSpace, ReadLatency, WriteLatency, DiskQueueDepth,
# ReplicaLag, MaximumUsedTransactionIDs, OldestReplicationSlotLag
```

Healthy looks like

CPU average under about 60% with peaks under 85%; FreeableMemory stable; free storage above 20%; latencies low and flat; no metric has a new step-change compared with yesterday.

If not

Pick the matching scenario: CPU (RT-01), connections (RT-02), storage (RT-06), replica lag (RT-07), I/O (RT-15), memory (RT-13), XID age (RT-09).

D-03 Backups are running and restorable

Purpose

Backups fail silently if nobody looks. `LatestRestorableTime` shows how far back and how recently point-in-time recovery works.

Run

```
aws rds describe-db-instances --db-instance-identifier <id> \
  --query "DBInstances[0].[BackupRetentionPeriod,PreferredBackupWindow,LatestRestorableTime]"

aws rds describe-db-snapshots --db-instance-identifier <id> --snapshot-type automated \
  --query "reverse(sort_by(DBSnapshots,&SnapshotCreateTime))[0:3].
           [DBSnapshotIdentifier,Status,SnapshotCreateTime]"
```

Healthy looks like

Retention is greater than 0, the newest automated snapshot is `available` and less than 24 h old, and `LatestRestorableTime` is within about 5 minutes of now.

If not

Stale snapshot or restorable time: check events for backup failures, free storage, and whether retention was set to 0 by mistake. Take a manual snapshot now if in doubt.

D-04 Connections and session states

Purpose

Connection exhaustion causes outages that look like 'the database is down'. Also spot connection leaks early.

Run

```
SELECT state, count(*) FROM pg_stat_activity
WHERE backend_type = 'client backend' GROUP BY state ORDER BY 2 DESC;

SHOW max_connections;

SELECT usename, application_name, count(*) FROM pg_stat_activity
WHERE backend_type = 'client backend' GROUP BY 1, 2 ORDER BY 3 DESC LIMIT 10;
```

Healthy looks like

Total connections under 80% of `max_connections`; very few `idle in transaction`; no application using an unexpectedly large share.

If not

Near the limit or many idle-in-transaction sessions: see RT-02 and RT-05.

D-05 Long-running queries, transactions and blocked sessions

Purpose

Old transactions hold locks and block vacuum; blocked sessions mean users are waiting right now.

Run

```
SELECT pid, usename, state, now() - xact_start AS xact_age,
       now() - query_start AS query_age, wait_event_type, wait_event, left(query, 70) AS q
FROM pg_stat_activity
WHERE state <> 'idle' AND backend_type = 'client backend' AND pid <> pg_backend_pid()
ORDER BY xact_start NULLS LAST LIMIT 15;

SELECT pid, pg_blocking_pids(pid) AS blocked_by, left(query, 60) AS q
FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

Healthy looks like

During business hours no transaction is older than a few minutes (known batch jobs excepted) and no session is blocked.

If not

Blocked sessions: RT-04. Old or idle-in-transaction sessions: RT-05.

D-06 Top SQL by total time

Purpose

The few statements that dominate total time decide your performance. Watching the top 10 daily reveals regressions and new heavy queries quickly. Also open Performance Insights, which shows the same by database load.

Run

```
SELECT queryid, calls,
       round(total_exec_time::numeric / 1000, 1) AS total_s,
       round(mean_exec_time::numeric, 2) AS mean_ms, rows, left(query, 60) AS q
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;
-- PostgreSQL 12 and older: use total_time / mean_time
```

Healthy looks like

The top consumers are the same as yesterday, with similar mean times.

If not

A new query in the top 5, or a mean time that doubled: RT-03.

D-07 Autovacuum and dead tuples

Purpose

Dead rows pile up when autovacuum falls behind, causing bloat, slower queries and eventually wraparound risk.

Run

```
SELECT relname, n_live_tup, n_dead_tup,
       round(100.0 * n_dead_tup / nullif(n_live_tup + n_dead_tup, 0), 1) AS dead_pct,
       last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 10;

SELECT pid, now() - xact_start AS runtime, left(query, 80) AS q
FROM pg_stat_activity WHERE query LIKE 'autovacuum:%';
```

Healthy looks like

Dead percentage under roughly 10 to 20% on large tables and `last_autovacuum` is recent for busy tables. Autovacuum workers appear and finish.

If not

High dead percentage or old `last_autovacuum`: RT-08.

D-08 Error log review

Purpose

The PostgreSQL log is the best early-warning source: FATAL/PANIC messages, deadlocks, temp-file spills, authentication failures and checkpoint warnings.

Run

```
aws rds describe-db-log-files --db-instance-identifier <id> \
  --query "DescribeDBLogFiles[-3:].[LogFileName,Size]"

aws rds download-db-log-file-portion --db-instance-identifier <id> \
  --log-file-name <log_file> --output text \
  | grep -E "FATAL|PANIC|ERROR|deadlock|temporary file|checkpoints are occurring"

# CloudWatch Logs Insights (if logs are exported):
# fields @timestamp, @message | filter @message like /FATAL|PANIC|deadlock detected/
# | sort @timestamp desc | limit 50
```

Healthy looks like

Only expected errors. No `PANIC`. No bursts of `too many connections` or `password authentication failed`.

If not

Investigate the pattern: new error types after a deploy usually point to application changes; auth bursts can mean a rotated secret or an attack.

D-09 Replication: read replicas and slots

Purpose

Lagging replicas serve stale data; inactive slots silently fill the disk with WAL.

Run

```
-- on the primary
SELECT client_addr, state, sync_state,
       pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS replay_lag_bytes
FROM pg_stat_replication;

SELECT slot_name, slot_type, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM pg_replication_slots;

-- on a read replica (also use the ReplicaLag metric, see RT-07)
SELECT now() - pg_last_xact_replay_timestamp() AS replica_delay;
```

Healthy looks like

Replica lag is seconds; every slot is `active` with small retained WAL.

If not

Growing lag: RT-07. Inactive slot retaining WAL: RT-14.

D-10 Storage and growth

Purpose

Running out of storage is the most preventable outage. Track database size daily and know your days-to-full.

Run

```
SELECT pg_size_pretty(pg_database_size(current_database())) AS db_size;

SELECT relname,
       pg_size_pretty(pg_total_relation_size(relid)) AS total,
       pg_size_pretty(pg_relation_size(relid))       AS heap
FROM pg_stat_user_tables ORDER BY pg_total_relation_size(relid) DESC LIMIT 10;

# CloudWatch: FreeStorageSpace and TransactionLogsDiskUsage trend
```

Healthy looks like

Growth matches expectation and days-to-full is above 30. Storage autoscaling is enabled with a sensible maximum.

If not

A sudden jump: RT-06 (find the consumer first, add storage second).

D-11	Alarms, pending maintenance, open tickets
Purpose	Close the loop: make sure nothing is firing, nothing is about to be changed under you, and handover items are tracked.

Run	`aws cloudwatch describe-alarms --state-value ALARM \`
	`  --query "MetricAlarms[].[AlarmName,StateUpdatedTimestamp]" --output table`
	
	`aws rds describe-pending-maintenance-actions \`
	`  --filters Name=db-instance-id,Values=<id>`

Healthy looks like	No alarms in `ALARM`; pending maintenance (if any) has a known date and approval.
If not	Reschedule or defer maintenance that collides with business peaks, and notify the application owners.

PART 2 | Weekly, monthly and quarterly activities

Recurring upkeep that stops small problems becoming incidents. Schedule these in the team calendar and keep the outputs as evidence.

W-01 WEEKLY: Capacity and growth review

Purpose

Turn daily numbers into a forecast: when will storage, connections or CPU become a problem? Keeping your own size history is cheap and answers this question precisely.

Run

```
CREATE SCHEMA IF NOT EXISTS dba;
CREATE TABLE IF NOT EXISTS dba.size_history (
  ts timestamptz DEFAULT now(), db text, bytes bigint);

-- run daily (cron or pg_cron)
INSERT INTO dba.size_history(db, bytes)
SELECT datname, pg_database_size(datname) FROM pg_database WHERE NOT datistemplate;

-- weekly growth
SELECT db, pg_size_pretty(max(bytes) - min(bytes)) AS growth_7d
FROM dba.size_history WHERE ts > now() - interval '7 days' GROUP BY db;
```

Healthy looks like

Weekly growth is in line with the forecast; peak connections under 70% of the limit; CPU peaks have headroom.

If not

Raise a capacity request: bigger instance class, more IOPS, higher autoscaling ceiling, or data archival/partitioning.

W-02 WEEKLY: Index review (unused, duplicate, missing)

Purpose

Every unused index slows writes and wastes storage; every missing index slows reads. Statistics are cumulative since the last reset or restart, so judge over a long window.

Run

```
-- unused (never scanned) indexes, largest first
SELECT schemaname, relname, indexrelname, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes JOIN pg_index USING (indexrelid)
WHERE idx_scan = 0 AND NOT indisunique AND NOT indisprimary
ORDER BY pg_relation_size(indexrelid) DESC LIMIT 20;

-- duplicate indexes
SELECT indrelid::regclass AS tbl, array_agg(indexrelid::regclass) AS duplicates
FROM pg_index
GROUP BY indrelid, indkey::text, indclass::text, indoption::text, indexprs::text, indpred::text
HAVING count(*) > 1;

-- candidates for a missing index: big tables read mostly by sequential scans
SELECT relname, seq_scan, seq_tup_read, idx_scan,
       pg_size_pretty(pg_relation_size(relid)) AS size
FROM pg_stat_user_tables
WHERE seq_scan > idx_scan AND pg_relation_size(relid) > 100 * 1024 * 1024
ORDER BY seq_tup_read DESC LIMIT 10;
```

Healthy looks like

A short list that you review with the application team. Drop candidates are confirmed unused on the primary and on every replica.

If not

Drop with `DROP INDEX CONCURRENTLY`; add with `CREATE INDEX CONCURRENTLY`. Never drop a unique/primary-key index, and keep a script to recreate anything you drop.

W-03 WEEKLY: Bloat check and autovacuum tuning

Purpose	Bloat means files are bigger than needed, so more I/O and cache misses. Find the worst tables and tune autovacuum for them specifically rather than changing it globally.

Run	`CREATE EXTENSION IF NOT EXISTS pgstattuple;`
	`SELECT * FROM pgstattuple_approx('<schema>.<table>');  -- dead_tuple_percent, free_percent`
	
	`-- make autovacuum more aggressive for one hot table`
	`ALTER TABLE <schema>.<table> SET (`
	`  autovacuum_vacuum_scale_factor = 0.02,`
	`  autovacuum_analyze_scale_factor = 0.01);`
	
	`-- reclaim space online (extension supported on RDS): run from a client`
	`-- pg_repack -h <endpoint> -U <user> -d <db> -t <schema>.<table>`

Healthy looks like	Large tables stay under roughly 20% bloat and vacuum runs finish within their window.
If not	Prefer `pg_repack` over `VACUUM FULL`; `VACUUM FULL` takes an exclusive lock and blocks the table for the whole rewrite.

W-04 WEEKLY: Slow-query tuning cycle

Purpose	A repeatable loop: pick the worst queries, understand the plan, fix, and measure.

Run	`-- 1) pick the top 5 from pg_stat_statements (D-06)`
	`-- 2) get the real plan`
	`EXPLAIN (ANALYZE, BUFFERS) <query>;`
	`-- 3) look for: Seq Scan on big tables, row estimate far from actual rows,`
	`--    'Sort Method: external merge  Disk', nested loops over millions of rows`
	`-- 4) fix (index, rewrite, statistics) then re-run EXPLAIN and compare`
	`CREATE INDEX CONCURRENTLY idx_<table>_<cols> ON <schema>.<table> (<cols>);`

Healthy looks like	Each tuned query has a recorded before/after time and plan.
If not	If a change does not help, drop the index you added and note the finding. Always test on a clone or in off-peak hours first.

W-05 WEEKLY: Access and security review

Purpose	Know who can do what. Remove stale accounts and confirm there are no surprise privileged roles.

Run	`SELECT r.rolname, r.rolcanlogin, r.rolcreaterole, r.rolcreatedb,`
	`       ARRAY(SELECT b.rolname FROM pg_auth_members m`
	`             JOIN pg_roles b ON b.oid = m.roleid WHERE m.member = r.oid) AS member_of`
	`FROM pg_roles r WHERE r.rolname !~ '^pg_' ORDER BY 1;`
	
	`SHOW rds.force_ssl;`
	`SHOW password_encryption;`

Healthy looks like	Only expected roles hold `rds_superuser`; SSL is enforced; passwords use `scram-sha-256`; security groups allow only known sources.
If not	Revoke or drop stale roles (reassign owned objects first), rotate any shared credentials, and review the CloudTrail/RDS log entries for unusual logins.

W-06 WEEKLY: Parameter and configuration drift

Purpose	Console clicks and emergency changes drift from what is documented. Compare reality with your baseline (or infrastructure-as-code).
Run	`aws rds describe-db-parameters --db-parameter-group-name <pg> --source user \` `  --query "Parameters[].[ParameterName,ParameterValue,ApplyMethod]" --output table`
Healthy looks like	The list of changed parameters matches the documented baseline.
If not	Record who changed what and why, and update the baseline or revert the change.

M-01 MONTHLY: Restore test (point-in-time recovery to a scratch instance)

Purpose	A backup is only proven by a restore. This also measures how long recovery really takes, which feeds your recovery-time objective.

Run	`aws rds restore-db-instance-to-point-in-time \`
	`  --source-db-instance-identifier <id> --target-db-instance-identifier <id>-restoretest \`
	`  --use-latest-restorable-time --db-instance-class <smaller_class> --no-multi-az`
	
	`aws rds wait db-instance-available --db-instance-identifier <id>-restoretest`
	
	`-- validate: row counts of critical tables, max(created_at), a few business queries`
	
	`aws rds delete-db-instance --db-instance-identifier <id>-restoretest \`
	`  --skip-final-snapshot --delete-automated-backups`

Healthy looks like	The restore completes in the expected time and validation queries return plausible, current data. The test instance is deleted afterwards.
If not	A restore from a large snapshot loads data lazily, so first reads are slow; note this in your recovery estimates.

M-02 MONTHLY: Minor version and maintenance planning

Purpose	Minor releases carry security and bug fixes. Plan them rather than letting them surprise you in the maintenance window.

Run	`aws rds describe-db-engine-versions --engine postgres --engine-version <current> \`
	`  --query "DBEngineVersions[].ValidUpgradeTarget[].[EngineVersion,IsMajorVersionUpgrade]" \`
	`  --output table`
	
	`aws rds describe-pending-maintenance-actions \`
	`  --filters Name=db-instance-id,Values=<id>`

Healthy looks like	A target minor version is chosen, tested in non-production, and scheduled with approval.
If not	See C-05 for the upgrade procedure.

M-03 MONTHLY: Failover drill (Multi-AZ)

Purpose	The only way to know your applications survive a failover is to cause one in a controlled window and watch.

Run	`aws rds reboot-db-instance --db-instance-identifier <id> --force-failover`
	
	`aws rds describe-events --source-type db-instance --source-identifier <id> --duration 60 \`
	`  --query "Events[].[Date,Message]" --output table`
	
	`SELECT pg_postmaster_start_time(), pg_is_in_recovery();`

Healthy looks like	Failover finishes in typically 1 to 2 minutes; applications reconnect on their own; no manual restarts are needed.
If not	If apps do not recover by themselves, fix reconnect/retry logic and DNS caching (keep the endpoint TTL honoured) before the next drill.

M-04 MONTHLY: CA certificate and TLS expiry

Purpose	Expired or rotated CA certificates break TLS connections suddenly, usually on a weekend.

Run	`aws rds describe-db-instances \`
	`  --query "DBInstances[].[DBInstanceIdentifier,CACertificateIdentifier]" --output table`
	
	`aws rds describe-certificates \`
	`  --query "Certificates[].[CertificateIdentifier,ValidTill]" --output table`

Healthy looks like	All instances use a CA with at least 6 months of validity, and application trust stores contain the current bundle.
If not	Plan the CA change, update client trust stores first, then modify the instance.

M-05	MONTHLY: Cost and right-sizing
Purpose	Right-size from evidence: sustained CPU and memory, not guesses. Remove forgotten snapshots and test instances.

Run	`aws rds describe-db-snapshots --snapshot-type manual \`
	`  --query "DBSnapshots[?SnapshotCreateTime<='2026-08-01'].`
	`  [DBSnapshotIdentifier,SnapshotCreateTime]" \`
	`  --output table`
	
	`# CloudWatch: 30-day average and p95 of CPUUtilization and FreeableMemory`

Healthy looks like	No orphaned snapshots/instances; instance class matches 30-day utilization; reserved capacity considered for steady workloads.
If not	Downsize or upsize in a planned window (C-03).

Q-01	QUARTERLY: Governance, DR and lifecycle
Purpose	Longer-cycle items that prevent slow-burning risk.

Run	`Quarterly checklist:`
	` - Engine version support/EOL review and next major-upgrade plan`
	` - Cross-region snapshot copy and a test restore in the DR region`
	` - Access recertification and secret/password rotation`
	` - Review RPO/RTO against the latest restore and failover test results`
	` - Review alarm thresholds against real incident history`
	` - Runbook walk-through with the on-call team`

Healthy looks like	Each item has an owner, a date and a recorded outcome.
If not	Open actions with owners; escalate anything past its due date.

PART 3 | Real-time scenarios (incident playbooks)

Each playbook follows the same flow: recognise the symptom, run the diagnostic commands, read the result, apply the fix, and then prevent a repeat. When several things are wrong at once, stabilise first (stop the damage, free the resource), then find the root cause. Always tell stakeholders what you are doing before taking an action that terminates sessions.

RT-01 CPU at 90 to 100%

Symptom

Alarm on CPUUtilization; application is slow or timing out.

Diagnose (run)

```
-- what is running right now (longest first)
SELECT pid, usename, state, now() - query_start AS runtime,
       wait_event_type, wait_event, left(query, 80) AS q
FROM pg_stat_activity WHERE state = 'active' AND pid <> pg_backend_pid()
ORDER BY query_start LIMIT 15;

-- cumulative heavy hitters
SELECT queryid, calls, round(mean_exec_time::numeric, 1) AS mean_ms,
       round(total_exec_time::numeric / 1000) AS total_s, left(query, 60) AS q
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 5;

# Performance Insights: DBLoad by SQL and by wait (CPU vs IO vs Lock)
```

How to read it

Many active sessions running the same query on CPU means a bad plan or a missing index. More active sessions than vCPUs means saturation. If DBLoad is mostly I/O or lock waits rather than CPU, the root cause is elsewhere (RT-15, RT-04). Check if it started right after a deployment.

Fix now

- Identify the offending query or job and cancel it (`pg_cancel_backend`, then `pg_terminate_backend` if needed).
- If caused by a release, roll it back or disable the feature.
- If the load is legitimate read traffic, add a read replica or scale up the instance.
- Fix the cause (index, query rewrite) after the incident.

Fix commands

```
SELECT pg_cancel_backend(<pid>);       -- graceful, cancels the query only
SELECT pg_terminate_backend(<pid>);    -- ends the session
```

Prevent

Query review before releases, a pg_stat_statements baseline, connection pooling, and an alarm at 80% CPU for 15 minutes so you act before 100%.

RT-02 Too many connections [HIGH IMPACT]

Symptom

Errors: 'FATAL: remaining connection slots are reserved for non-replication superuser connections' or 'sorry, too many clients already'. Application cannot connect.

Diagnose (run)

```
SELECT count(*) AS total,
       count(*) FILTER (WHERE state = 'idle')                AS idle,
       count(*) FILTER (WHERE state = 'idle in transaction') AS idle_in_tx,
       count(*) FILTER (WHERE state = 'active')              AS active
FROM pg_stat_activity WHERE backend_type = 'client backend';

SELECT usename, application_name, client_addr, count(*)
FROM pg_stat_activity GROUP BY 1, 2, 3 ORDER BY 4 DESC LIMIT 10;

SHOW max_connections;
```

How to read it

Mostly `idle`: application pools are oversized or leaking. Mostly `idle in transaction`: an application bug that does not commit. A spike from one host: a runaway client or restart loop. Remember: total pool size across ALL application instances must stay below `max_connections`.

Fix now

- Connect with the master user (reserved slots exist for rds_superuser).
- Terminate idle sessions of the offending application.
- Ask the owner to shrink/restart the misbehaving pool.
- Do not just raise `max_connections`: it is a static parameter (reboot), and every connection costs memory.

Fix commands

```
SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE usename = '<app_role>' AND state = 'idle'
  AND state_change < now() - interval '10 minutes';
```

Prevent

Use RDS Proxy or PgBouncer; set `ALTER ROLE <app> CONNECTION LIMIT n`; set `idle_in_transaction_session_timeout` and `idle_session_timeout` (PG 14+); alarm at 80% of `max_connections`.

RT-03 One query suddenly became slow (plan regression)

Symptom

A report or API call that took 200 ms now takes 30 s, starting yesterday or after a deployment.

Diagnose (run)

```
EXPLAIN (ANALYZE, BUFFERS) <the slow query>;

SELECT relname, n_mod_since_analyze, last_analyze, last_autoanalyze
FROM pg_stat_user_tables WHERE relname = '<table>';

SELECT queryid, calls, round(mean_exec_time::numeric, 1) AS mean_ms,
       round(stddev_exec_time::numeric, 1) AS stddev_ms
FROM pg_stat_statements WHERE query LIKE '%<fragment>%';
```

How to read it

- Estimated rows far from actual rows: stale or insufficient statistics.
- Seq Scan on a large table where an index should be used: index missing/dropped, or the planner changed its mind.
- 'Sort Method: external merge Disk': `work_mem` too small for this query.
- Nested loop over millions of rows: plan flip caused by bad estimates.

Also check what changed: deployment, parameter change, big data load, version upgrade.

Fix now

- Run `ANALYZE` on the involved tables first (cheap and often enough).
- Create the missing index with `CONCURRENTLY`.
- For correlated columns, create extended statistics.
- Use session-level `SET` of planner settings only to prove the cause, not as the fix.

Fix commands

```
ANALYZE <schema>.<table>;
CREATE STATISTICS st_<name> (dependencies) ON <col1>, <col2> FROM <schema>.<table>;
ALTER TABLE <schema>.<table> ALTER COLUMN <col> SET STATISTICS 500;
ANALYZE <schema>.<table>;
```

Prevent

Capture plans of top queries (auto_explain, Performance Insights history), keep autovacuum analyze healthy, and run ANALYZE after large loads and after upgrades.

RT-04 Lock pile-up: sessions waiting on a blocker [HIGH IMPACT]

Symptom

Many sessions stuck, application timeouts, `wait_event_type = Lock` in pg_stat_activity.

Diagnose (run)

```
-- blocked sessions and who blocks them
SELECT a.pid, a.usename, a.state, now() - a.xact_start AS xact_age,
       pg_blocking_pids(a.pid) AS blocked_by, left(a.query, 60) AS q
FROM pg_stat_activity a WHERE cardinality(pg_blocking_pids(a.pid)) > 0;

-- the ROOT blocker: blocks others but is not blocked itself
SELECT DISTINCT b AS root_blocker
FROM (SELECT unnest(pg_blocking_pids(pid)) AS b FROM pg_stat_activity) x
WHERE NOT EXISTS (SELECT 1 FROM pg_stat_activity y
                  WHERE y.pid = x.b AND cardinality(pg_blocking_pids(y.pid)) > 0);

-- what is it (state, query, owner)?
SELECT pid, usename, state, now() - xact_start AS xact_age, query
FROM pg_stat_activity WHERE pid = <root_blocker_pid>;
```

How to read it

The root blocker is very often an `idle in transaction` session that took a lock and never finished, or a long batch job. A frequent trap: an `ALTER TABLE` waits for an ACCESS EXCLUSIVE lock behind a long query, and then every new query queues behind the ALTER. Cancelling the waiting DDL frees the queue immediately.

Fix now

- Find the root blocker, its owner and its query.
- If it is idle in transaction or safe to stop, cancel/terminate it.
- If a queued DDL is the cause, cancel the DDL.
- Tell the application owner so it is fixed, otherwise it returns.

Fix commands

```
SELECT pg_cancel_backend(<pid>);       -- try first
SELECT pg_terminate_backend(<pid>);    -- if it does not release
```

Prevent

Use `SET lock_timeout = '5s'` before DDL, keep transactions short, set `idle_in_transaction_session_timeout`, run DDL off-peak, and prefer `CONCURRENTLY` operations.

RT-05 Idle-in-transaction and very old transactions

Symptom

Vacuum is not cleaning, tables bloat, or locks are held by sessions doing nothing.

Diagnose (run)

```
SELECT pid, usename, application_name, client_addr,
       now() - xact_start AS xact_age, state, left(query, 60) AS last_query
FROM pg_stat_activity
WHERE state IN ('idle in transaction', 'idle in transaction (aborted)')
ORDER BY xact_start LIMIT 20;
```

How to read it

An open transaction pins the oldest snapshot (`xmin`). Vacuum cannot remove any row version newer than that, so bloat and XID age grow database-wide, and any locks it holds keep blocking others.

Fix now

- Terminate sessions that are clearly abandoned.
- Ask the owning team to fix missing commit/rollback or connection handling.
- Put timeouts in place so it cannot recur.

Fix commands

```
SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE state = 'idle in transaction' AND now() - xact_start > interval '15 minutes';

ALTER ROLE <app_role> SET idle_in_transaction_session_timeout = '5min';
```

Prevent

Set the timeout per application role, monitor the count of idle-in-transaction sessions, and review ORM/pool transaction handling.

RT-06 Free storage dropping fast or storage full [HIGH IMPACT]

Symptom

FreeStorageSpace alarm, instance status `storage-full`, or errors such as 'No space left on device'. Writes may fail.

Diagnose (run)

```
# CloudWatch: FreeStorageSpace, TransactionLogsDiskUsage, OldestReplicationSlotLag

-- 1) WAL held by replication slots
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM pg_replication_slots;

-- 2) temp-file spills
SELECT datname, temp_files, pg_size_pretty(temp_bytes) AS temp_bytes
FROM pg_stat_database ORDER BY temp_bytes DESC;

-- 3) big or ballooning tables
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS total
FROM pg_stat_user_tables ORDER BY pg_total_relation_size(relid) DESC LIMIT 10;
```

How to read it

The usual suspects, in order: an inactive replication slot retaining WAL (RT-14), huge temp files from sorts/hashes, a runaway load or table/index bloat, large logs, or plain data growth.

Fix now

- First buy time: increase allocated storage (online; the new size must be at least 10% larger, and further changes are blocked for about 6 hours).
- Then remove the cause: drop a confirmed-abandoned slot, stop the runaway query, purge old data in batches followed by vacuum.
- Do not delete data in one huge transaction: it also generates WAL and bloat.

Fix commands

```
aws rds modify-db-instance --db-instance-identifier <id> \
  --allocated-storage <new_GiB> --apply-immediately

aws rds modify-db-instance --db-instance-identifier <id> \
  --max-allocated-storage <ceiling_GiB>          # enable/raise autoscaling
```

Prevent

Enable storage autoscaling with a ceiling, alarm at 20% and 10% free, watch slots, set `max_slot_wal_keep_size` and `temp_file_limit`.

RT-07 Read replica lag keeps growing

Symptom

ReplicaLag alarm; users on the replica see stale data.

Diagnose (run)

```
-- on the replica
SELECT pg_is_in_recovery(), now() - pg_last_xact_replay_timestamp() AS replay_delay;
SELECT pg_last_wal_receive_lsn() AS received, pg_last_wal_replay_lsn() AS replayed;
SELECT datname, confl_snapshot, confl_lock, confl_bufferpin
FROM pg_stat_database_conflicts;

-- on the primary
SELECT client_addr, state,
       pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS replay_lag_bytes
FROM pg_stat_replication;

# CloudWatch: ReplicaLag, primary WriteIOPS, replica CPU and IOPS
```

How to read it

- `received` is current but `replayed` is behind: the replica cannot apply fast enough (CPU/I-O bound, or long queries conflicting with replay).
- `received` itself is behind: network or a write burst on the primary.

Gotcha: `now() - pg_last_xact_replay_timestamp()` keeps growing on an idle primary even though nothing is missing. Trust the `ReplicaLag` metric and the LSN gap.

Fix now

- Cancel long-running queries on the replica.
- Scale the replica class/IOPS to match the primary.
- Break up big write batches on the primary.
- Route long analytics to a dedicated replica.

Fix commands

```
SELECT pg_cancel_backend(pid) FROM pg_stat_activity
WHERE state = 'active' AND now() - query_start > interval '5 minutes'
  AND pid <> pg_backend_pid();
```

Prevent

Keep replicas the same size as the primary, alarm on ReplicaLag (30 s / 300 s), and avoid `hot_standby_feedback` unless you accept extra bloat on the primary.

RT-08 Autovacuum cannot keep up: dead tuples and bloat

Symptom

Dead percentage climbing, table and index sizes growing, queries slowing, `last_autovacuum` old or never.

Diagnose (run)

```
-- who holds back vacuum? (oldest xmin holders)
SELECT pid, state, age(backend_xmin) AS xmin_age, now() - xact_start AS xact_age,
       left(query, 50) AS q
FROM pg_stat_activity WHERE backend_xmin IS NOT NULL
ORDER BY age(backend_xmin) DESC LIMIT 5;

SELECT slot_name, active, age(xmin) AS xmin_age, age(catalog_xmin) AS catalog_xmin_age
FROM pg_replication_slots;

SELECT gid, prepared, age(transaction) AS xid_age FROM pg_prepared_xacts;
```

How to read it

Vacuum can only remove rows older than the oldest snapshot. The holder is one of: an old transaction, a stale replication slot, a replica with `hot_standby_feedback`, or a forgotten prepared transaction. If none exist, autovacuum is simply too slow for the table (too little cost limit or scale factor too large for a huge table).

Fix now

- Remove the xmin holder first, otherwise vacuuming achieves nothing.
- Run a manual vacuum on the worst tables in a quiet period.
- Make autovacuum more aggressive for those tables.
- Reclaim space with pg_repack, not VACUUM FULL, if the table is on a live system.

Fix commands

```
VACUUM (VERBOSE, ANALYZE) <schema>.<table>;

ALTER TABLE <schema>.<table> SET (
  autovacuum_vacuum_scale_factor = 0.01,
  autovacuum_vacuum_cost_limit   = 2000);
```

Prevent

Per-table autovacuum settings for hot, large tables; `log_autovacuum_min_duration` to see what vacuum does; alarms on dead-tuple percentage.

RT-09 Transaction ID wraparound warning [HIGH IMPACT]

Symptom

MaximumUsedTransactionIDs is high and climbing, or the log says 'database must be vacuumed within N transactions'. In the worst case PostgreSQL stops accepting writes.

Diagnose (run)

```
SELECT datname, age(datfrozenxid) AS xid_age FROM pg_database ORDER BY 2 DESC;

SELECT c.oid::regclass AS tbl, age(c.relfrozenxid) AS xid_age
FROM pg_class c WHERE c.relkind IN ('r','m','t') ORDER BY 2 DESC LIMIT 10;

SHOW autovacuum_freeze_max_age;
-- plus the xmin-holder queries from RT-08
```

How to read it

Normal operation keeps age below `autovacuum_freeze_max_age` (default 200 million) because autovacuum freezes old rows. If age keeps rising far above that, vacuum is blocked (old transaction/slot/prepared transaction) or too slow. The hard limit is about 2 billion; treat anything above 1 billion as an emergency.

Fix now

- Remove xmin holders (RT-08).
- Vacuum with freeze on the oldest tables first, one by one.
- Do not interrupt these vacuums; give the session more `maintenance_work_mem`.
- If it is close to the limit and not dropping, open an AWS Support case immediately.

Fix commands

```
SET maintenance_work_mem = '1GB';
VACUUM (FREEZE, VERBOSE) <schema>.<oldest_table>;
```

Prevent

Alarm on MaximumUsedTransactionIDs (500 million warning / 1 billion critical), schedule off-peak freeze vacuums for huge static tables, and keep autovacuum healthy.

RT-10 Unexpected failover or reboot [HIGH IMPACT]

Symptom	Connections dropped for 1 to 2 minutes, applications logged errors, RDS shows a failover or reboot event.

Diagnose (run)	`aws rds describe-events --source-type db-instance --source-identifier <id> \`
	`  --duration 720 --query "Events[].[Date,Message]" --output table`
	
	`SELECT pg_postmaster_start_time(), now() - pg_postmaster_start_time() AS uptime;`
	
	`aws rds download-db-log-file-portion --db-instance-identifier <id> \`
	`  --log-file-name <log_file_near_event> --output text \`
	`  | grep -E "PANIC|terminated by signal|server process|out of memory"`

How to read it	Events tell you whether it was a planned reboot, maintenance, an infrastructure problem (host/AZ), or a database crash. 'terminated by signal 9' or out-of-memory entries, plus a FreeableMemory collapse just before the event in CloudWatch/Enhanced Monitoring, indicate the OS killed PostgreSQL for memory.
Fix now	• Confirm applications reconnected; restart any that did not. • Expect a performance dip while caches warm up. • Re-run D-01 to D-05 and confirm replicas are healthy. • Write an RCA from events, logs and metrics.
Prevent	Retry/reconnect logic with short timeouts in applications, memory discipline (connections x work_mem), alarms on FreeableMemory and SwapUsage, and regular failover drills (M-03).

RT-11 Accidental DELETE, UPDATE or DROP: recover the data [HIGH IMPACT]

Symptom	A user ran a destructive statement on production. Need the data back with minimal impact.

Diagnose (run)	`-- 0) If the session is still inside its transaction: ROLLBACK; (best case).`
	`-- 1) Stop further damage, note the exact UTC time just BEFORE the mistake.`
	`-- 2) Restore to a NEW instance (RDS never overwrites the original)`
	`aws rds restore-db-instance-to-point-in-time \`
	`  --source-db-instance-identifier <id> --target-db-instance-identifier <id>-pitr \`
	`  --restore-time 2026-10-29T09:55:00Z --db-instance-class <class> --no-multi-az`
	
	`aws rds wait db-instance-available --db-instance-identifier <id>-pitr`
	
	`-- 3) Copy only the lost rows back (from a client host)`
	`pg_dump -h <pitr_endpoint> -U <user> -d <db> -t <schema>.<table> --data-only \`
	`  | psql -h <prod_endpoint> -U <user> -d <db>`

How to read it	Point-in-time recovery rebuilds the instance as it was at that moment, so compare it with production and copy back only what is missing. Load into a staging table first and merge with `INSERT ... ON CONFLICT DO NOTHING` to avoid duplicates. The restore time must be within the backup retention window and before `LatestRestorableTime`.
Fix now	• Restore, verify the data is present, merge it back, validate with the business. • Delete the restored instance when finished. • For a dropped table, the same process applies: dump the table from the restored instance.
Prevent	Least-privilege access, `BEGIN;` before manual DML and check the row count before `COMMIT;`, deletion protection on instances, and long enough backup retention.

RT-12 Deadlocks

Symptom	Application errors with SQLSTATE 40P01, log entries 'deadlock detected'.

Diagnose (run)	`SELECT datname, deadlocks FROM pg_stat_database ORDER BY deadlocks DESC;`
	
	`# log file (see D-08): find the DETAIL lines showing both processes and statements`
	`aws rds download-db-log-file-portion --db-instance-identifier <id> \`
	`  --log-file-name <log_file> --output text | grep -A8 "deadlock detected"`

How to read it	Two transactions each hold a lock the other needs. PostgreSQL aborts one of them. The log shows both statements and the tables involved. Typical causes: updating rows in different orders, long transactions, and foreign keys without supporting indexes.
Fix now	• Make all code paths update tables and rows in the same order. • Keep transactions short. • Add indexes on foreign-key columns. • Make the application retry on 40P01.
Prevent	Set `log_lock_waits = on`, review `deadlock_timeout`, and add deadlock count to dashboards.

RT-13 Memory pressure: low FreeableMemory, swap, temp-file spills

Symptom	FreeableMemory falling, SwapUsage rising, queries slowing, log lines such as 'temporary file: path ... size ...'.

Diagnose (run)	`SELECT datname, temp_files, pg_size_pretty(temp_bytes) AS temp_bytes`
	`FROM pg_stat_database ORDER BY temp_bytes DESC;`
	
	`SHOW work_mem; SHOW max_connections; SHOW shared_buffers;`
	
	`-- in the plan of a suspect query look for 'Disk:' on Sort/Hash nodes`
	`EXPLAIN (ANALYZE, BUFFERS) <query>;`

How to read it	Worst-case memory is roughly connections x `work_mem` x the number of sort/hash steps per query, on top of shared buffers. Too small a `work_mem` makes queries spill to disk (slow, I/O heavy); too large combined with many connections risks the OS killing PostgreSQL (see RT-10).
Fix now	• Do not raise `work_mem` globally by a large factor. • Set it per role or per session for the heavy reporting users. • Reduce the number of connections with pooling. • Add indexes so sorts are not needed; scale memory if the working set truly grew.

Fix commands	`ALTER ROLE <report_role> SET work_mem = '128MB';`
	`-- ALTER SYSTEM is not available on RDS; change it in the parameter group:`
	`aws rds modify-db-parameter-group --db-parameter-group-name <pg> \`
	`  --parameters "ParameterName=log_temp_files,ParameterValue=10240,ApplyMethod=immediate"`

Prevent	Alarms on FreeableMemory/SwapUsage, Enhanced Monitoring enabled, and a connection budget per application.

RT-14 Inactive replication slot is retaining WAL [HIGH IMPACT]

Symptom	TransactionLogsDiskUsage or OldestReplicationSlotLag keeps growing and free storage falls, although data volume is flat.

Diagnose (run)	`SELECT slot_name, plugin, slot_type, active, restart_lsn,`
	`       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained`
	`FROM pg_replication_slots ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;`
	
	`SHOW max_slot_wal_keep_size;`

How to read it	A slot makes PostgreSQL keep all WAL the consumer has not yet confirmed. If the consumer (DMS task, Debezium, logical subscriber) is stopped or deleted but the slot remains, WAL grows until storage is full.
Fix now	• If the consumer is still needed, restart it quickly so it catches up. • If it is gone for good, drop the slot. Dropping a slot that is still needed means the consumer must be re-initialised, so confirm with its owner first.
Fix commands	`SELECT pg_drop_replication_slot('<slot_name>');`
Prevent	Alarm on OldestReplicationSlotLag, set `max_slot_wal_keep_size`, and make slot clean-up part of decommissioning any CDC/migration tool.

RT-15 High I/O latency or DiskQueueDepth

Symptom	ReadLatency/WriteLatency up, DiskQueueDepth rising, queries slow but CPU is not the problem.

Diagnose (run)	`# CloudWatch: ReadIOPS/WriteIOPS vs provisioned, ReadLatency, WriteLatency, DiskQueueDepth,`
	`# BurstBalance (gp2), EBSIOBalance% / EBSByteBalance% (burstable instance classes)`
	
	`SELECT queryid, shared_blks_read, shared_blks_hit, left(query, 60) AS q`
	`FROM pg_stat_statements ORDER BY shared_blks_read DESC LIMIT 10;`
	
	`SELECT round(100.0 * sum(blks_hit) / nullif(sum(blks_hit + blks_read), 0), 2) AS cache_hit_pct`
	`FROM pg_stat_database;`

How to read it	• Low cache-hit ratio with heavy reads: queries scanning big tables (missing indexes) or memory too small for the working set. • Heavy writes: bulk loads, index builds, big vacuums, checkpoint bursts. • IOPS or throughput at the provisioned limit (or burst credits exhausted) means the storage or instance is the ceiling.
Fix now	• Stop or throttle the heavy job. • Fix the top read-heavy queries. • Increase provisioned IOPS/throughput, or move to a larger instance class if the instance EBS bandwidth is the limit. • Schedule batch work off-peak.
Fix commands	`aws rds modify-db-instance --db-instance-identifier <id> \` `  --storage-type gp3 --iops <iops> --storage-throughput <MiBps> --apply-immediately`
Prevent	Know your baseline IOPS, alarm on latency, and test batch jobs off-peak first.

RT-16 Cannot connect to the database

Symptom	Application or psql fails with timeout, refused, or authentication errors.

Diagnose (run)	`aws rds describe-db-instances --db-instance-identifier <id> \`
	`  --query "DBInstances[0].[DBInstanceStatus,Endpoint.Address,PubliclyAccessible]"`
	
	`nslookup <endpoint>`
	`nc -zv <endpoint> 5432`
	`psql "host=<endpoint> dbname=<db> user=<user> sslmode=require" -c "select 1"`

How to read it	• Instance not `available` (rebooting, `storage-full`): wait or fix the state. • DNS fails: resolver or private-zone problem. • Timeout on port 5432: security group, NACL, route table, or public-access setting. • 'password authentication failed': wrong or recently rotated credentials. • 'no pg_hba.conf entry ... no encryption': SSL is enforced; use `sslmode=require` or `verify-full`. • 'too many connections': RT-02. TLS errors after a CA change: M-04.
Fix now	Work from the network layer upwards: status, DNS, port, TLS, credentials, limits. Fix the first failing layer and retest.
Prevent	Test connectivity from each application subnet in advance, store credentials in Secrets Manager, and document the security-group model.

RT-17 Parameter change did not take effect

Symptom	You changed a parameter but `SHOW <param>` still returns the old value.

Diagnose (run)	`SHOW <param>;`
	
	`aws rds describe-db-instances --db-instance-identifier <id> \`
	`  --query "DBInstances[0].DBParameterGroups"  # look for ParameterApplyStatus`
	
	`aws rds describe-db-parameters --db-parameter-group-name <pg> \`
	`  --query "Parameters[?ParameterName=='<param>'].[ParameterName,ParameterValue,ApplyType]"`

How to read it	`ApplyType` is `dynamic` (applies without restart) or `static` (needs a reboot). A `pending-reboot` apply status means the group is changed but the engine is still running the old values. Also check that the instance really uses the group you edited.
Fix now	• Dynamic parameter: apply immediately. • Static parameter: schedule a reboot in a window (with failover if Multi-AZ). • Re-check with `SHOW` after the reboot.

Fix commands	`aws rds modify-db-parameter-group --db-parameter-group-name <pg> \`
	`  --parameters \`
	`  "ParameterName=log_min_duration_statement,ParameterValue=500,ApplyMethod=immediate"`
	
	`aws rds reboot-db-instance --db-instance-identifier <id>  # for static parameters`

Prevent	Test parameter changes on a clone, record them in change management, and avoid editing the default parameter group.

RT-18 'permission denied' for a new or existing user

Symptom	A newly created user or a new table gives 'permission denied for table/schema/sequence'.

Diagnose (run)	`SELECT has_database_privilege('<role>', '<db>', 'CONNECT');`
	`SELECT has_schema_privilege('<role>', '<schema>', 'USAGE');`
	`SELECT has_table_privilege('<role>', '<schema>.<table>', 'SELECT');`
	`SELECT has_sequence_privilege('<role>', '<schema>.<seq>', 'USAGE');`
	
	`SELECT * FROM pg_default_acl;  -- default privileges for future objects`
	`\dp <schema>.<table>  -- psql: table privileges`

How to read it	Access needs the whole chain: CONNECT on the database, USAGE on the schema, then the table/sequence privilege. New tables are not covered by earlier `GRANT ... ON ALL TABLES`; that is what default privileges are for. From PostgreSQL 15 the `public` schema no longer lets everyone create objects.
Fix now	• Grant the missing link in the chain. • Add `ALTER DEFAULT PRIVILEGES` so future tables are covered. • Remember default privileges apply to objects created by the role that set them (or named with `FOR ROLE`).

Fix commands	`GRANT USAGE ON SCHEMA <schema> TO <role>;`
	`GRANT SELECT ON ALL TABLES IN SCHEMA <schema> TO <role>;`
	`ALTER DEFAULT PRIVILEGES FOR ROLE <owner_role> IN SCHEMA <schema>`
	`  GRANT SELECT ON TABLES TO <role>;`

Prevent	Use group roles (`app_ro`, `app_rw`) and grant membership instead of per-user grants (see C-01).

PART 4 | Routine change tasks

Standard procedures for the changes a DBA is asked to make regularly. Take a snapshot before anything risky and prefer the maintenance window for changes that restart the instance.

C-01 Create application roles (read-only and read-write)

When

A new application or user needs database access.

Steps / commands

```
-- as the master user (rds_superuser); use a password from Secrets Manager
CREATE ROLE app_ro NOLOGIN;
CREATE ROLE app_rw NOLOGIN;
GRANT CONNECT ON DATABASE <db> TO app_ro, app_rw;
GRANT USAGE ON SCHEMA app TO app_ro, app_rw;
GRANT SELECT ON ALL TABLES IN SCHEMA app TO app_ro;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA app TO app_rw;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA app TO app_rw;
ALTER DEFAULT PRIVILEGES IN SCHEMA app GRANT SELECT ON TABLES TO app_ro;
ALTER DEFAULT PRIVILEGES IN SCHEMA app
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_rw;
ALTER DEFAULT PRIVILEGES IN SCHEMA app GRANT USAGE, SELECT ON SEQUENCES TO app_rw;

CREATE ROLE svc_orders LOGIN PASSWORD '<secret>' CONNECTION LIMIT 50 IN ROLE app_rw;
ALTER ROLE svc_orders SET statement_timeout = '30s';
ALTER ROLE svc_orders SET idle_in_transaction_session_timeout = '5min';
```

Verify

`SELECT has_table_privilege('svc_orders','app.<table>','INSERT');` returns true, and the application can connect over SSL.

Watch out

The master user is not an operating-system superuser; it is a member of `rds_superuser`. Never share the master credentials with applications.

C-02 Change a parameter safely

When

Tuning or enabling logging; static parameters need a reboot.

Steps / commands

```
1. Check the current value:            SHOW <param>;
2. Test the change on a clone or non-production instance.
3. Apply to the parameter group:
aws rds modify-db-parameter-group --db-parameter-group-name <pg> \
  --parameters "ParameterName=<param>,ParameterValue=<value>,ApplyMethod=immediate"
4. For static parameters, schedule a reboot (use ApplyMethod=pending-reboot):
aws rds reboot-db-instance --db-instance-identifier <id>
5. Verify:                              SHOW <param>;
```

Verify

`SHOW` returns the new value and `ParameterApplyStatus` is `in-sync`.

Watch out

A wrong value in a static parameter can stop the instance from starting (`incompatible-parameters`). Change one thing at a time and keep the previous value noted.

C-03 Scale instance class, storage or IOPS

When

Capacity change after W-01/M-05 review or during an incident.

Steps / commands

```
1. Take a manual snapshot first.
aws rds create-db-snapshot --db-instance-identifier <id> \
  --db-snapshot-identifier pre-change-<date>

2. Instance class (causes a restart; schedule a window; Multi-AZ usually shortens the outage):
aws rds modify-db-instance --db-instance-identifier <id> --db-instance-class <new> \
  --apply-immediately

3. Storage size / type / IOPS (online):
aws rds modify-db-instance --db-instance-identifier <id> --allocated-storage <GiB> \
  --storage-type gp3 --iops <iops> --apply-immediately

aws rds describe-db-instances --db-instance-identifier <id> \
  --query "DBInstances[0].[DBInstanceStatus,PendingModifiedValues]"
```

Verify

Status returns to `available`, `PendingModifiedValues` is empty, and CloudWatch shows the new capacity in effect. Run D-01 to D-05.

Watch out

Storage can grow but never shrink, and a new storage change is blocked for about 6 hours after the previous one. Size for the next 6 to 12 months, not just today.

C-04 Create and use a read replica

When

Offloading reporting/read traffic or building a DR copy.

Steps / commands

```
aws rds create-db-instance-read-replica --db-instance-identifier <id>-replica \
  --source-db-instance-identifier <id> --db-instance-class <class>

aws rds wait db-instance-available --db-instance-identifier <id>-replica

-- on the replica
SELECT pg_is_in_recovery();                      -- true

-- DR only: promote (breaks replication, becomes standalone and writable)
aws rds promote-read-replica --db-instance-identifier <id>-replica
```

Verify

Replica is `available`, `ReplicaLag` is low, and the application's reporting connection string points at the replica endpoint.

Watch out

A replica is asynchronous; reads can be slightly stale. Promotion is one-way. Long queries on the replica can delay replay (RT-07).

C-05 Minor version upgrade

When

Security and bug-fix releases within the same major version.

Steps / commands

```
1. Read the release notes; test on a restored clone first.
2. Snapshot: aws rds create-db-snapshot --db-instance-identifier <id> \
     --db-snapshot-identifier pre-minor-<date>
3. Schedule for the next maintenance window (or immediately):
aws rds modify-db-instance --db-instance-identifier <id> --engine-version <target> \
  --no-apply-immediately
4. After the restart, verify:
SELECT version();
5. Update extensions if the release notes say so:
ALTER EXTENSION <name> UPDATE;
```

Verify

`version()` shows the target, applications are healthy, and replicas follow. Run D-01 to D-09.

Watch out

There is a short outage; with Multi-AZ it is usually about a minute or two. Upgrade replicas as the release notes advise and keep application retry logic in place.

C-06 Major version upgrade (Blue/Green recommended)

When

Moving to a newer major version. Blue/Green builds a synchronized copy on the new version, so the switch-over is short and you can test beforehand.

Steps / commands

```
-- pre-checks on the current instance
SELECT slot_name, slot_type FROM pg_replication_slots;   -- logical slots may block upgrades
SELECT gid FROM pg_prepared_xacts;                       -- must be empty
SELECT extname, extversion FROM pg_extension;            -- supported in the target?

aws rds describe-db-engine-versions --engine postgres --engine-version <current> \
  --query "DBEngineVersions[].ValidUpgradeTarget[].[EngineVersion,IsMajorVersionUpgrade]"

-- Blue/Green
aws rds create-blue-green-deployment --blue-green-deployment-name <name> \
  --source <instance_arn> --target-engine-version <new_version>
-- test the green environment, then switch over
aws rds switchover-blue-green-deployment --blue-green-deployment-identifier <bgd-id> \
  --switchover-timeout 300

-- after the switch
vacuumdb --all --analyze-in-stages -h <endpoint> -U <user>
```

Verify

Application tests pass on the green copy before switch-over; afterwards query plans and timings match the baseline and statistics are rebuilt.

Watch out

Statistics are not carried across a major upgrade, so run ANALYZE straight away. Test extension compatibility and application drivers beforehand, and take a snapshot first.

C-07 Logical backup and data export

When	A portable copy of the data, or moving data to another system.

Steps / commands	`# run from an EC2 host in the same VPC (fast and cheap)`
	`pg_dump -h <endpoint> -U <user> -d <db> -Fd -j 4 -f /backup/<db>_$(date +%F)`
	
	`pg_restore -h <target> -U <user> -d <db> -j 4 /backup/<db>_<date>`
	
	`# snapshot export to S3 (Parquet), no load on the database`
	`aws rds start-export-task --export-task-identifier <name> --source-arn <snapshot_arn> \`
	`  --s3-bucket-name <bucket> --iam-role-arn <role_arn> --kms-key-id <kms_key>`

Verify	Restore completes without errors and row counts of critical tables match.
Watch out	`pg_dump` takes a consistent snapshot but holds an old transaction for its duration, which delays vacuum on very busy databases. Use a replica for long dumps.

C-08 Rotate credentials

When	Scheduled rotation, or after a suspected leak.

Steps / commands	`-- application role`
	`ALTER ROLE svc_orders PASSWORD '<new_secret>';`
	
	`-- master user`
	`aws rds modify-db-instance --db-instance-identifier <id> \`
	`  --master-user-password '<new_secret>' --apply-immediately`
	
	`-- check the hashing method (should be scram-sha-256)`
	`SHOW password_encryption;`

Verify	Applications reconnect with the new secret, and old credentials fail. Secrets Manager holds the new value.
Watch out	Rotate secrets in the application and the database in the right order (applications should read the secret dynamically). Prefer Secrets Manager automatic rotation or IAM database authentication where possible.

PART 5 | Quick answers to common questions

RDS fundamentals

Question	Answer
What do I manage and what does AWS manage?	AWS manages hardware, OS and host patching, automated backups, Multi-AZ failover mechanics and engine binaries. You manage schema, queries, indexes, vacuum health, parameters, users and grants, capacity planning, monitoring, restore testing and how applications connect.
Can I SSH into the server or be a superuser?	No. There is no OS access and no true superuser. You get the master user, which is a member of `rds_superuser`. Use CloudWatch, Enhanced Monitoring and Performance Insights for visibility.
Multi-AZ versus read replica: what is the difference?	Multi-AZ is for availability: a standby in another AZ with synchronous replication and automatic failover; you cannot read from a Multi-AZ instance standby. A read replica is for scaling reads: asynchronous, readable, and can be promoted manually.
How long does a Multi-AZ failover take, and is data lost?	Typically 1 to 2 minutes, as the endpoint DNS moves to the standby. Committed transactions are not lost because replication to the standby is synchronous. Applications must reconnect and retry.
How is `max_connections` decided?	By a formula based on instance memory unless you set it in the parameter group. Bigger instance classes allow more connections, but each connection uses memory, so use pooling (RDS Proxy or PgBouncer) instead of just raising it.
Where are the logs?	In the console and via `aws rds download-db-log-file-portion`. Export them to CloudWatch Logs for retention and search. Logging detail is controlled by parameters such as `log_min_duration_statement`, `log_lock_waits`, `log_temp_files`.

Daily operations

Question	Answer
`pg_cancel_backend` versus `pg_terminate_backend`?	`pg_cancel_backend(pid)` cancels the current query but keeps the session. `pg_terminate_backend(pid)` ends the whole session and rolls back its open transaction. Try cancel first.
What does 'idle in transaction' mean and why is it harmful?	The session started a transaction and is waiting without committing. It holds locks and its snapshot, so it blocks other sessions and stops vacuum from cleaning dead rows. Terminate it and set `idle_in_transaction_session_timeout`.
How do I find what is blocking a query?	Use `pg_blocking_pids(pid)` in `pg_stat_activity` (see RT-04), find the root blocker, and cancel or terminate it after understanding what it is doing.
Why is my table the same size after a big DELETE?	Deleted rows become dead tuples; vacuum makes the space reusable inside the table but normally does not return it to the operating system. To shrink the file use `pg_repack` (online) or `VACUUM FULL` (exclusive lock).
VACUUM, VACUUM FULL and autovacuum: what is the difference?	Autovacuum is the background process that runs vacuum and analyze automatically. `VACUUM` makes dead space reusable without blocking normal work. `VACUUM FULL` rewrites the table, returns space to the OS, and blocks all access while it runs.
How do I find unused indexes safely?	Query `pg_stat_user_indexes` where `idx_scan = 0`, excluding unique and primary-key indexes, over a long observation window and on replicas too (W-02). Drop with `DROP INDEX CONCURRENTLY` and keep the definition to recreate it.

Performance

Question	Answer
The query was fast yesterday. Where do I start?	Run `EXPLAIN (ANALYZE, BUFFERS)`, check whether estimated and actual rows differ, check `last_autoanalyze` for the tables, and look for recent deployments, parameter changes or data growth (RT-03). `ANALYZE` fixes a surprising share of these.
How do I enable and use `pg_stat_statements`?	On RDS the library is normally preloaded; run `CREATE EXTENSION pg_stat_statements;` in the database and query the view ordered by `total_exec_time`. Performance Insights gives a graphical view of the same data.
Why are queries slow right after a restore or upgrade?	The cache is cold, restored volumes load blocks lazily, and after a major upgrade planner statistics are missing. Run `ANALYZE` (for example `vacuumdb --analyze-in-stages`) and allow time for warm-up.
Should I raise `work_mem` when I see temp files?	Not globally. Memory use multiplies by connections and plan nodes. Raise it per role or session for the heavy workloads, and also consider indexes that avoid the sort.

Backup, recovery and upgrades

Question	Answer
Can point-in-time recovery overwrite my existing instance?	No. It always creates a new instance. Recover the data from it, or repoint the application to it if appropriate.
How do I restore just one dropped table?	Do a point-in-time restore to a new instance just before the drop, dump the table from it and load it into production (RT-11), then delete the restored instance.
How can I reduce downtime for a major upgrade?	Use RDS Blue/Green Deployments: a synchronized copy runs the new version, you test it, then switch over in typically about a minute. Always run ANALYZE afterwards.
What happens when storage fills up?	The instance can become `storage-full` and writes fail. Increase allocated storage immediately (online), then remove the cause. Enable autoscaling with a maximum to avoid this.
Why was a storage change rejected?	After a storage modification there is a waiting period of about 6 hours before another, and increases must be at least 10%. Plan sizes with headroom.

Security

Question	Answer
How do I force encrypted connections?	Set the parameter `rds.force_ssl = 1`, use `sslmode=verify-full` with the RDS CA bundle in clients, and track CA expiry (M-04).
What is the best way to store database credentials?	AWS Secrets Manager with automatic rotation, or IAM database authentication. Use `scram-sha-256` password hashing and separate least-privilege roles per application.

PART 6 | Command cheat sheet

psql meta-commands

Command	Use
`\l \c <db> \dn`	List databases, switch database, list schemas
`\dt+ <schema>.*`	Tables with sizes
`\d+ <table>`	Columns, indexes, constraints, triggers, size
`\du \dp <table>`	Roles; table privileges
`\x auto \timing on`	Expanded output when wide; show query time
`\watch 5`	Re-run the last query every 5 seconds
`\copy tbl TO 'f.csv' CSV HEADER`	Client-side export (works on RDS)

Handy SQL

Command	Use
`SELECT pg_size_pretty(pg_database_size(current_database()));`	Database size
`SELECT now() - pg_postmaster_start_time();`	Uptime (reveals restarts)
`SELECT pg_cancel_backend(pid); / pg_terminate_backend(pid);`	Cancel query / end session
`SELECT pg_reload_conf();`	Not needed on RDS: use parameter groups
`SELECT * FROM pg_stat_progress_vacuum;`	Watch a running vacuum
`SELECT * FROM pg_stat_progress_create_index;`	Watch an index build
`VACUUM (VERBOSE, ANALYZE) t; ANALYZE t;`	Clean up and refresh statistics
`CREATE INDEX CONCURRENTLY ... REINDEX INDEX CONCURRENTLY ...`	Online index work

AWS CLI (RDS)

Command	Use
`aws rds describe-db-instances`	Status, class, storage, Multi-AZ, pending changes
`aws rds describe-events --duration <min>`	Recent events (max 14 days)
`aws rds describe-db-log-files / download-db-log-file-portion`	List and read logs
`aws rds modify-db-instance ... --apply-immediately`	Scale, change settings
`aws rds modify-db-parameter-group`	Change parameters
`aws rds reboot-db-instance [--force-failover]`	Reboot, optionally with failover
`aws rds create-db-snapshot / describe-db-snapshots`	Manual snapshot / list
`aws rds restore-db-instance-to-point-in-time`	Point-in-time restore to a new instance
`aws rds create-db-instance-read-replica / promote-read-replica`	Replica lifecycle
`aws rds describe-pending-maintenance-actions`	Upcoming maintenance